

SUMMER INDUSTRY TRAINING PROJECT REPORT

ON

“STOCK VISION”

AT

O7 SERVICES

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF DEGREE

MASTER OF COMPUTER APPLICATIONS

Submitted To:

Mr. Deepak Kumar

Assistant Professor (SCS)

Submitted By:

Pooja

GU-2024-1740



SCHOOL OF COMPUTATIONAL SCIENCE, GNA UNIVERSITY

SRI HARGOBINDGARH, PHAGWARA-HOSHIARPUR ROAD,

PHAGWARA-144401, PUNJAB, INDIA.

BATCH 2024 - 2026

AUGUST 2025 TO DECEMBER 2025

SUMMER INDUSTRY TRAINING PROJECT REPORT

ON

“STOCK VISION”

AT

O7 SERVICES

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF DEGREE

MASTER OF COMPUTER APPLICATIONS

Submitted To:

Mr. Deepak Kumar

Assistant Professor (SCS)

Submitted By:

Pooja

GU-2024-1740



SCHOOL OF COMPUTATIONAL SCIENCE, GNA UNIVERSITY

SRI HARGOBINDGARH, PHAGWARA-HOSHIARPUR ROAD,

PHAGWARA-144401, PUNJAB, INDIA.

BATCH 2024 - 2026

AUGUST 2025 TO DECEMBER 2025

CANDIDATE’S DECLARATION

I hereby declare that the project entitled “**STOCK VISION**” submitted by **Pooja (GU-2024-1740)** for partial fulfillment of the requirement for the award of degree of **MCA (Masters of Computer Application)** submitted in SCS, GNA University is an authentic record of work carried out during a period from Aug, 2025 to Dec, 2025. The matter presented in this project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles

Signature of the Student

Place:

Date:

COMPANY PROFILE

O7 SERVICES Pvt Ltd, Jalandhar.



O7 Services is an ISO 9001:2015 certified company, established in 2015, with a strong reputation for delivering end-to-end IT solutions. The company specializes in a wide range of services, including Web Development, Mobile Application Development, Custom Software Development, UI/UX Design, Hosting Services, Digital Marketing, Domain Registration across multiple extensions, AMC & MMC services, Bulk SMS, and Voice Call solutions.

O7 Services has developed a wide range of software products, including Vehicle Tracking Systems, Invoice Software, School and Hospital Management Systems, Parent-Teacher Communication Apps, Fee and Task Management Systems, Online Food Ordering Applications, Security Apps, Admission Systems, Inventory Software, and Car Servicing Apps. Each solution is designed to be practical, reliable, and user-friendly, addressing real-world business needs effectively.

O7 Services also offers industry-focused training programs, including 6-week and 6-month industrial training, project-based training, corporate training, and job-oriented courses, all tailored to meet current industry needs. The training covers a broad spectrum of modern technologies, such as Full Stack Development (MEAN/MERN), Flutter, Kotlin Android, Swift UI (iOS), Firebase, Python, Angular, React.js, Vue.js, Node.js, ASP.NET, .NET Core, PHP, Laravel, CodeIgniter, Software Testing, Cloud Computing, Blockchain, DevOps, Data Science, Graphic Designing, Artificial Intelligence, Machine Learning, UI/UX Design, Digital Marketing, WordPress, Linux, CCNP, CCNA Security, Network Security, Cyber Security, Ethical Hacking, MCSE, MCITP, Java, Spring, Hibernate, C/C++, Photoshop, Adobe Illustrator, Figma, CorelDRAW, AI Bot and more.

 +91- 8437365007, +91-181-5015007

 enquiry@o7services.com , hr@o7services.com

 www.o7services.com



TRAINING COMPLETION CERTIFICATE



ABSTRACT

Stock Vision is a machine-learning powered financial analytics system designed to assist retail investors in making informed decisions by analyzing historical stock performance, technical indicators, and future price forecasts. Traditional trading decisions often rely on guesswork, emotional bias, or scattered data sources, which can lead to poor investment choices and financial losses. This project tackles that problem by providing a reliable, data-driven platform that helps users evaluate market trends with accuracy and confidence.

The system uses historical market data fetched through the yfinance API, including vital parameters such as Open, High, Low, Close, Volume, moving averages, volatility patterns, and momentum indicators. Using this cleaned and preprocessed dataset, a Long Short-Term Memory (LSTM) deep learning model is trained to forecast short-term stock prices. The model achieves strong predictive performance by using a 60-day historical window and is capable of generating 2-day, 7-day, and 30-day future price predictions. Alongside LSTM, additional technical indicators such as Bollinger Bands, MACD, volume analysis, and return distributions are integrated for deeper insights.

To enhance real-time relevance, the system retrieves live stock market data automatically, ensuring up-to-date price movements and technical updates. The entire solution is deployed as a Streamlit web application, where users can select default tickers or enter custom symbols. The dashboard allows investors to visualize price charts, candlesticks, volume trends, and predictive graphs instantly. A clean interface with loader animations ensures smooth user experience while long operations like model loading or data fetching are processed.

Stock Vision demonstrates how machine learning and data science can enhance modern investing by offering powerful, intuitive, and AI-driven decision-support tools. The system helps users reduce investment risk, understand market behavior, and make smarter financial decisions through real-time data analysis, robust forecasting, and interactive visual insights all in one unified platform.

ACKNOWLEDGEMENT

The successful completion of this project marks the beginning of an ever - going learning experience of converting ideas and concepts into real life, practical system. This project was a quite a learning experience for me at each and every step. At the same time, it has given me confidence to work in professional setup. I feel the experience gained during the project will lead me to gain the bright prospect in the future. First of all, I would like to give thanks to **Head, Education and Training O7 Services, Mr. Gaurav Sharma**, for giving me the opportunity to work in this esteemed organization, which not only has increased our awareness about latest fields but also taught me the importance of team building. With the deep sense of gratitude, I express my sincere thanks to **Dr. Hitesh Marwaha, HOD, School of Computational Sciences GNA University, Phagwara**, for stimulating guidance, continuous encouragement and supervision throughout the course of present work.

Pooja

GU-2024-1740

MCA 3rd Sem

Signature of the HoD/Dean,SCS

TABLE OF CONTENT

Content	Page No.
Candidate's Declaration	iii
Company Profile	iv
Training Competition Certificate	vi
Abstract	vii
Acknowledgement	viii
Table of content	ix
List of Figures	xi
Chapter 1: Introduction	
1.1 Stock Vision	1
1.2 Problem Statement	2
1.3 Problem Specification	2 - 4
1.4 Project Purpose	4 -5
1.5 Module Description	6 - 7
1.6 Benefits of this Project	7
1.7 Software Description	8
Chapter 2: System Analysis	
2.1 Existing System	9 - 10
2.2 Lacks of Existing System	10
2.3 Proposed System	11 - 12
2.4 Feasibility	12 - 14
2.5 Feasibility Conclusion	14
Chapter 3: Development Environment	
3.1 Functional Requirements	15 - 16

3.2 Hardware Requirements	16
3.3 Software Requirements	16
3.4 Technology Stack Used	16 - 17
Chapter 4: System Design	
4.1 Requirement Specification	18 - 19
4.2 Flowchart	19 - 21
4.3 Data Flow Diagram	22 - 23
4.4 Component Design	23 - 24
4.5 Entity Relationship Diagram	25 - 27
Chapter 5: System Implementation	
5.1 Methodology	28 - 29
5.2 Technology Used	30 - 31
5.3 Testing	32 - 33
5.4 Implementation at User Level	33 - 34
5.5 Application Screenshots	35 - 37
Chapter 6: Results and Discussions	
6.1 Result	38
Chapter 7: Conclusion	
7.1 Conclusion	39
7.2 Future Scope	39 - 40
Chapter 8: References	
8.1 References	41

LIST OF FIGURES

TITLE	PAGE NUMBER
4.2.1 FLOWCHART FOR STOCK VISION	19
4.3.1 DATA FLOW DIAGRAM (LEVEL 0)	22
4.3.3 DATA FLOW DIAGRAM (LEVEL 1)	23
4.4.1 COMPONENTS OF THE STOCK VISION	25
4.5.1 ENTITY RELATIONSHIP DIAGRAM	27
5.5.1 HOME PAGE	35
5.5.2 VISUAL ANALYSIS PAGE	36
5.5.3 NEWS PAGE	36
5.5.4 PREDICTION PAGE	37

CHAPTER 1

INTRODUCTION

1.1 STOCK VISION

The stock market plays a crucial role in shaping the economic growth of modern nations, influencing investment decisions, corporate development, and overall financial stability. As more individuals and retail traders participate in financial markets, the need for intelligent, technology-driven tools has become essential. Traditional trading methods depend heavily on personal intuition, fragmented information, and speculative decision-making, often leading to inconsistent outcomes and financial risks. To navigate the complexities of the stock market, investors increasingly require data-backed insights and predictive analytics to make smarter, more informed trading decisions.

Earlier, stock selection and price prediction were primarily guided by historical experience, market rumors, manual chart analysis, and general assumptions about market trends. However, stock performance is influenced by numerous technical and statistical factors such as price movements, volatility, trading volume, momentum indicators, moving averages, and investor sentiment. Without proper analytical tools or knowledge of technical patterns, many investors struggle to interpret market behavior effectively.

To address these challenges, Stock Vision leverages Machine Learning and technical analysis to provide accurate, data-driven insights for stock selection and price forecasting. Machine Learning, a powerful subset of artificial intelligence, enables large-scale financial data analysis to identify patterns, correlations, and market signals that are impossible to detect manually. With modern computational tools and deep learning architectures, ML has become a transformative force in algorithmic trading and financial decision-making.

In this system, key stock market features such as historical prices (Open, High, Low, Close), volume, returns, moving averages, Bollinger Bands, MACD, and other technical indicators are collected using reliable data sources like *yfinance*. These inputs are preprocessed and used to train an LSTM-based deep learning model, which demonstrates strong predictive ability for short-term stock price forecasting. Based on the market behavior, Stock Vision provides insights through interactive visualizations and prediction graphs, helping users understand trends, evaluate stock strength, and make informed trading decisions.

1.2 PROBLEM STATEMENT

Retail investors often struggle to make informed decisions in the stock market due to limited knowledge of technical analysis, lack of consistent data access, and the overwhelming complexity of market fluctuations. Traditional investment decisions are frequently based on intuition, market rumors, emotional reactions, or basic price observations approaches that rarely align with actual market behavior. As a result, investors frequently buy or sell stocks at the wrong time, leading to financial losses, poor portfolio performance, and increased market risk.

Additionally, many investors do not have access to real-time stock data, advanced analytical tools, or predictive models capable of interpreting historical trends. Continuous market volatility, sudden price changes, and unpredictable movements further complicate decision-making. Without proper insights into indicators such as moving averages, volume trends, volatility patterns, and technical signals, investors find it difficult to evaluate stock strength or future potential. With growing participation in financial markets, the need for reliable, data-driven tools has become more crucial than ever.

Therefore, there is a need for an intelligent system that can analyze key market parameters such as historical prices, trading volume, momentum indicators, trend patterns, and volatility to generate accurate insights and price forecasts. The absence of such a system results in uninformed investment actions, increased risk exposure, and inconsistent financial outcomes for users.

Stock Vision aims to address this challenge by leveraging machine learning and deep learning models to deliver accurate analysis, technical visualizations, and short-term price predictions helping investors make smarter, research-backed financial decisions.

1.3 PROBLEM SPECIFICATION

Stock Vision is designed to overcome the limitations of traditional stock market analysis by providing a scientific, data-driven approach to evaluating stock performance and forecasting future price movements. The system must process multiple market factors such as historical prices, volume, volatility trends, moving averages, and technical indicators, as these variables heavily influence stock behavior. It must ensure accurate data preprocessing, including cleaning, normalization, feature extraction, and time-series preparation to maintain high predictive accuracy. Additionally, the system should address challenges related to rapid market

fluctuations, inconsistent investor knowledge, and the lack of accessible analytics tools for retail traders. By integrating machine learning, deep learning (LSTM), and real-time data retrieval, the platform aims to deliver reliable, user-friendly, and actionable stock insights that support informed investment decisions.

Detailed Specifications

• Efficient Data Handling

The system must efficiently collect, clean, and process large volumes of historical market data to ensure high accuracy and consistent model performance.

• Feature Engineering

Important features such as OHLC prices, volume, moving averages, Bollinger Bands, MACD, and daily returns must be properly extracted and analyzed to improve the accuracy and effectiveness of predictions.

• Machine Learning & Deep Learning Implementation

The system should implement robust models such as LSTM networks for time-series forecasting to generate precise short-term stock price predictions.

• Model Evaluation

Evaluation metrics such as RMSE, MAE, and prediction error percentages must be calculated to verify model reliability, performance, and forecasting quality.

• Real-Time Data Integration

The system must integrate APIs (e.g., yfinance or other market feeds) to automatically fetch the latest stock prices and market indicators for real-time analysis.

• User-Friendly Interface

A Streamlit-based interface should allow users to select tickers, adjust date ranges, and explore technical indicators effortlessly.

• Clear Visualization

The interface must display charts, technical dashboards, candlestick graphs, and prediction plots in a clean, intuitive, and interactive visual format.

- **Model Caching & Storage**

The system should store trained LSTM models and scalars efficiently so that they can be reused without retraining, ensuring faster performance.

- **Full Deployment Support**

The application must run smoothly on Streamlit Cloud, delivering high accessibility, low latency, and reliable performance across different devices.

- **Scalability**

The architecture should support future features such as portfolio optimization, sentiment analysis, RSI/ATR indicators, and multi-stock comparison modules.

- **Error Handling**

The system must efficiently manage invalid tickers, API failures, missing values, and inconsistent date ranges to ensure uninterrupted operation.

- **Security Measures**

Any user interactions, preferences, or stored configurations must be protected against unauthorized access and misuse.

- **Performance Optimization**

Data preprocessing, API usage, and LSTM computations must be optimized for fast execution, reduced latency, and smooth user experience.

1.4 PROJECT PURPOSE

The purpose of Stock Vision is to provide investors and financial enthusiasts with accurate, data-driven insights to improve stock analysis, price forecasting, and decision-making. By analyzing historical price trends, technical indicators, and real-time market data, the system helps minimize investment risks, reduce emotional trading, and enhance overall financial performance. The platform aims to bridge the gap between traditional stock analysis methods and modern AI-driven market intelligence by offering real-time predictions, intuitive visualizations, and a user-friendly interface. Ultimately, the project seeks to support informed investing, improve portfolio outcomes, and empower users with intelligent tools that enhance confidence, accuracy, and efficiency in stock market decisions.

Project Purpose

Accurate Stock Forecasting

Provide users with reliable short-term price predictions based on historical data, technical indicators, and market trends.

Technical Analysis Support

Assist investors by visualizing key indicators such as moving averages, volume trends, volatility bands, and candlestick patterns.

Real-Time Market Insights

Integrate live stock data to improve the accuracy, relevance, and timeliness of predictions and analytics.

Reduce Investment Risks

Help users avoid poor trading decisions by relying on data-driven insights instead of speculation or emotional judgment.

User-Friendly Platform

Offer a simple, interactive Streamlit-based interface that allows even beginners to analyze market trends effortlessly.

Promote Smart Investing

Encourage informed decision-making through AI-powered models, clean data visualization, and technical indicator interpretations.

Improve Profitability & Market Confidence

Enable users to make smarter investment choices that increase returns, minimize losses, and improve long-term financial outcomes.

Support Technology Adoption in Finance

Promote the use of machine learning, automation, and predictive analytics in retail trading and financial analysis.

Future Scalability

Build a foundation that can expand to include portfolio optimization, sentiment analysis, risk profiling, and multi-stock comparison features.

1.5 MODULE DESCRIPTION

The **Stock Vision** system is designed with multiple interconnected modules that work together to collect stock data, analyze trends, process machine learning models, and present actionable results to the user. Each module performs a specific function to ensure smooth system performance, accurate predictions, and a seamless user experience. The modular architecture makes the system easy to maintain, scalable for future enhancements, and flexible enough to integrate features such as sentiment analysis, portfolio optimization, and automated trading signals. The descriptions of all major modules are given below.

Data Collection Module: This module collects historical stock prices, trading volume, technical indicators, and live market data from APIs to ensure accurate and continuous data gathering.

- **Data Collection Module :** This module gathers historical stock prices, technical indicators, company data, and market information using APIs such as yfinance to ensure reliable and updated input data.
- **Data Preprocessing Module :** This module cleans, formats, and transforms raw financial data by handling missing values, adjusting date formats, and scaling numerical features for improved model performance.
- **Feature Engineering Module :** This module extracts and creates meaningful stock-related features such as moving averages, RSI, MACD, volatility measures, and trend indicators to enhance prediction accuracy.
- **Machine Learning Module :** This module trains models such as Random Forest Regressor and other predictive algorithms to forecast future stock prices and market trends.
- **Model Evaluation Module :** This module evaluates model performance using RMSE, MAE, and R^2 metrics to ensure the predictions are reliable, stable, and accurate.
- **Visualization Module :** This module displays price trends, technical indicator charts, candlestick patterns, and prediction outputs using clear and visually appealing graphs.
- **User Interface Module :** This module provides a clean, interactive Streamlit interface where users can search stocks, view charts, analyze indicators, and visualize predictions easily.
- **API Integration Module :** This module connects the system with financial data APIs like yfinance to fetch live and historical stock data seamlessly.
- **Prediction Module :** This module processes user-selected stock data and generates forecasted stock values for upcoming days using trained machine learning models.

- **Error Handling Module** : This module manages issues like invalid stock symbols, missing data, API failures, or model errors to maintain smooth application performance.
- **Security Module** : This module safeguards API keys, prevents unauthorized access, and ensures secure handling of all user and system data.
- **Deployment Module** : This module manages deployment on Streamlit Cloud or another hosting platform, ensuring smooth access, fast loading, and consistent performance.

1.6 BENEFITS OF THIS PROJECT

- **Accurate Stock Analysis** : Provides precise, data-driven insights based on historical prices, technical indicators, and machine learning predictions.
- **Improved Investment Decisions** : Helps users choose better stocks by analyzing trends, patterns, and market signals scientifically.
- **Reduced Financial Risk** : Minimizes losses by preventing guesswork and supporting trades with reliable analytical evidence.
- **Efficient Resource Utilization** : Saves time and effort by automating technical analysis, indicator generation, and prediction tasks.
- **Real-Time Market Support** : Uses live APIs to deliver up-to-date stock prices, indicators, and market trends for accurate decisions.
- **Cost-Effective Trading** : Helps users avoid wrong trades and improve profitability through predictive insights and trend analysis.
- **User-Friendly Interface** : Provides a clean and interactive Streamlit-based platform suitable for beginners, learners, and professionals.
- **Better Market Understanding** : Enhances financial literacy through clear explanations of technical indicators like RSI, MACD, and SMA.
- **Price Prediction Support** : Uses machine learning models to forecast stock movements and guide better entry and exit strategies.
- **Clear Visualization** : Presents charts, candlestick patterns, line graphs, and indicator plots in an easy-to-understand visual format.
- **High Scalability** : Supports future upgrades such as portfolio optimization, sentiment analysis, and automated trading signals.
- **Supports Digital Finance** : Encourages technology-based trading and investment, making the financial decision-making process smarter and more reliable.

1.7 SOFTWARE DESCRIPTION

- **Python:** The main programming language used to develop the entire system, including data processing, technical analysis, machine learning, API integration, and backend logic.
- **Pandas:** Used for cleaning, manipulating, and analyzing historical stock price data, including feature generation and time-series handling.
- **NumPy:** Provides numerical computing capabilities for calculations involving arrays, indicators, and mathematical operations.
- **Scikit-Learn:** Offers machine learning algorithms for stock price prediction, trend forecasting, preprocessing, and model evaluation.
- **Joblib:** Used to save and load trained machine learning models efficiently for deployment inside the Streamlit application.
- **Plotly:** Generates interactive and visually appealing charts such as candlesticks, line graphs, technical indicator plots, and prediction curves.
- **Streamlit:** A fast and interactive web framework used to build the user interface for real-time stock analysis, visualization, and prediction.
- **Requests Library:** Fetches real-time stock and market data from financial APIs to ensure live updates and accurate technical analysis.
- **Yahoo Finance API (yfinance):** Provides historical and real-time stock information, including open, high, low, close prices and volume data.
- **Streamlit-Option-Menu:** Used for creating clean navigation menus to enhance the user experience inside the application.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

In the current financial environment, most investors rely on personal experience, intuition, or generalized market advice rather than data-driven insights. Existing platforms often provide limited information, fragmented tools, and lack intelligent decision support, making it difficult for users to make informed trading or investment decisions. These systems rarely integrate advanced technologies like machine learning or predictive analytics, resulting in less accurate forecasts and unreliable trading signals.

Additionally, current solutions do not offer a unified platform where users can access historical stock data, technical indicators, real-time market prices, trend analysis, and price predictions in a single interface. The absence of real-time data integration, poor visualization, and limited automation creates challenges in making timely and profitable financial decisions. As a result, investors face increased risk, inefficient strategies, and missed opportunities in stock trading.

Existing System

- **Traditional Approaches :** Investors depend heavily on personal experience or generic advice rather than analytical insights.
- **No Real-Time Data Support :** Platforms do not provide dynamic updates for stock prices or market indicators.
- **Limited Technical Analysis :** Existing tools lack integrated charts, candlestick patterns, or technical indicator generation.
- **Generalized Advice :** Recommendations are broad and not tailored to specific stocks or market conditions.
- **Low Accuracy :** Absence of predictive models leads to unreliable price forecasts.
- **Poor User Interface :** Interfaces are often cluttered, non-interactive, and not suitable for beginners.
- **No API Integration :** Real-time market feeds and live stock updates are rarely incorporated.
- **High Manual Dependency :** Users must manually track stock trends, requiring significant time and effort.
- **No Multilingual Support :** Most platforms do not offer localized language options for global investors.

- **No Market Insight :** Existing solutions often fail to provide predictive trends, portfolio guidance, or actionable investment insights.

2.2 LACK OF EXISTING SYSTEM

Existing financial platforms and tools exhibit several gaps that limit their usefulness for modern investors. Most systems fail to provide real-time stock data, predictive analysis, or technical insights in a single, unified interface. Users are often required to manually collect historical data, compute indicators, and analyze trends without any automated support.

Many existing solutions lack machine learning-based forecasting, meaning investors cannot reliably predict short-term or long-term stock movements. Furthermore, visualization capabilities are limited, with charts and indicators often being static, non-interactive, or insufficiently informative.

Other key deficiencies include the absence of real-time API integration, poor scalability, and inadequate user experience, making it difficult for both beginners and professionals to make quick, informed decisions. Most platforms also do not provide portfolio insights, risk assessment, or actionable recommendations, leaving investors vulnerable to losses due to guesswork or delayed information.

Additional limitations are:

- **Limited Technical Indicator Support:** Most tools offer only basic indicators like SMA or EMA but lack a complete suite (RSI, MACD, Bollinger Bands) integrated in a single view.
- **No Prediction Visualization:** Existing platforms rarely combine prediction outputs with interactive charts or candlestick patterns.
- **Poor Historical Analysis Tools:** Users cannot efficiently analyze past trends to inform future investment decisions.
- **No Automation or Alerts:** Manual monitoring is required for stock price changes, trend reversals, or signals, increasing the risk of missed opportunities.
- **Lack of User Customization:** Few systems allow users to select stocks, timeframes, or technical metrics for personalized analysis.
- **Weak Data Security:** User data, preferences, or API keys are often not handled securely, risking privacy and data integrity.

2.3 PROPOSED SYSTEM

The Stock Vision system is designed to overcome the limitations of existing financial platforms by providing a unified, intelligent, and interactive stock analysis and prediction solution. It leverages historical data, real-time market feeds, technical indicators, and machine learning models to generate accurate insights and actionable recommendations for investors. By integrating multiple modules into a single platform, Stock Vision ensures efficient decision-making, enhanced user experience, and reduced financial risk.

The proposed system addresses the gaps in traditional tools and provides a modern, technology-driven approach to stock analysis. Key components and features include:

- **Unified Data Integration:** Combines historical stock data, live market feeds, and API-driven updates to provide complete, real-time information.
- **Machine Learning-Based Predictions:** Uses LSTM and other predictive models to forecast short-term and long-term stock price movements with high accuracy.
- **Comprehensive Technical Analysis:** Supports multiple indicators like SMA, EMA, RSI, MACD, Bollinger Bands, candlestick patterns, and trend analysis in a single interface.
- **Interactive Visualization:** Displays price trends, indicators, prediction outputs, and candlestick charts using Plotly for clear and user-friendly insights.
- **User-Friendly Interface:** Streamlit-based front-end allows easy stock selection, date range customization, and real-time interaction with charts and predictions.
- **Automated Alerts and Insights:** Notifies users of potential price trends, trend reversals, or significant market changes to support timely investment decisions.
- **Portfolio Monitoring:** Enables users to track selected stocks, analyze trends, and visualize potential returns for better financial planning.
- **Scalability and Flexibility:** Architecture supports the addition of advanced features such as sentiment analysis, automated trading signals, and portfolio optimization in the future.
- **Secure Data Handling:** User inputs, preferences, and API keys are safely stored and managed to ensure privacy and integrity.
- **Performance Optimization:** Efficient data preprocessing, caching, and model deployment ensure fast execution, minimal latency, and smooth system operation.

The Stock Vision system bridges the gap between traditional trading approaches and modern digital finance tools. By combining real-time data, predictive analytics, and interactive visualizations, it empowers investors to make informed decisions, minimize risks, and maximize profitability.

2.4 FEASIBILITY STUDY

A feasibility study evaluates the practicality and viability of the **Stock Vision** project in terms of technical, operational, economic, and schedule considerations. It ensures that the system can be developed efficiently, deployed effectively, and deliver measurable value to investors and financial enthusiasts.

1. Technical Feasibility

Stock Vision leverages modern, proven technologies to ensure successful development and deployment:

- **Programming Languages & Frameworks:** Python is used for backend logic, machine learning, and data processing. Streamlit provides an interactive front-end for visualization and user interaction.
- **Machine Learning Models:** LSTM (Long Short-Term Memory) networks are suitable for time-series forecasting, allowing accurate short-term and medium-term stock predictions. Additional models like Random Forest and Linear Regression can complement predictive accuracy.
- **Data Sources:** Yahoo Finance (yfinance), Alpha Vantage, and other APIs provide reliable historical and real-time stock data. Data cleaning, normalization, and preprocessing ensure model stability.
- **Visualization Tools:** Plotly and Matplotlib create interactive, responsive, and visually appealing charts, including candlestick plots, line graphs, and indicator overlays.
- **Scalability & Deployment:** The system can be hosted on Streamlit Cloud or similar cloud platforms, enabling multi-user access, low-latency operations, and easy scalability.
- **Security Measures:** Data protection practices, secure API key management, and controlled backend storage ensure integrity and privacy.

2. Operational Feasibility

The system is designed to meet operational needs and enhance user experience:

- **User-Friendly Interface:** The Streamlit-based interface allows intuitive navigation, easy stock selection, and interactive visualization for both beginners and experts.
- **Automation & Efficiency:** Automatic retrieval of live stock prices, technical indicator calculations, and predictive outputs reduce manual work and errors.
- **Accessibility:** Users can access Stock Vision from multiple devices (desktop, tablet, mobile), providing flexibility and convenience.
- **Real-Time Support:** Live API integration ensures that investors receive updated market data, improving timely decision-making.
- **Customizable Analysis:** Users can select specific stocks, timeframes, and indicators for personalized insights.
- **Alert Mechanisms:** Future implementation of notifications for trend reversals, price breakouts, or unusual market movements enhances operational efficiency.

3. Economic Feasibility

Stock Vision provides a cost-effective solution with measurable financial benefits:

- **Low Development Costs:** Utilizes open-source tools such as Python, Streamlit, and Plotly, reducing software expenses.
- **Minimal Hardware Requirements:** Can run on standard laptops or cloud-based platforms, eliminating the need for expensive infrastructure.
- **Time & Effort Savings:** Automates manual stock tracking, technical analysis, and prediction, saving considerable user effort.
- **Value Addition:** Improves investment accuracy, reduces potential losses, and provides actionable insights for better portfolio decisions.
- **Potential Monetization:** Future features like premium analytics, subscription-based insights, and automated trading tools could generate revenue.

4. Schedule Feasibility

- **Modular Development:** Core modules (data collection, preprocessing, visualization, ML prediction) can be developed concurrently to reduce project timeline.
- **Agile Approach:** Iterative development and testing allow faster implementation and timely delivery of features.

- **Time Estimation:** With a structured approach, the project can be completed in 8–12 weeks, including data collection, model training, interface design, and deployment.
- **Maintenance & Upgrades:** Post-deployment updates (new indicators, stock coverage, API integrations) can be integrated without major redesign.

5. Legal and Ethical Feasibility

- **Data Compliance:** Uses publicly available APIs and adheres to their terms of service.
- **Privacy:** No sensitive personal data is collected, and API keys are secured.
- **Transparency:** Predictive models are explainable, providing users with insights into forecasts and reducing misinformation risks.

2.5 FEASIBILITY CONCLUSION

The feasibility study demonstrates that the **Stock Vision** project is highly viable across technical, operational, economic, and schedule aspects. Technically, the system leverages mature and reliable tools such as Python, Streamlit, Plotly, and machine learning models like LSTM, ensuring accurate stock predictions and smooth performance. Operationally, the platform provides a user-friendly interface, real-time market updates, and interactive visualizations that simplify financial analysis for beginners and professionals alike. Economically, the project is cost-effective, requiring minimal hardware and software investment while delivering significant value by improving investment decisions and reducing financial risks.

From a scheduling and deployment perspective, Stock Vision is feasible to implement within a reasonable timeframe using a modular and agile development approach. The platform is scalable, allowing future enhancements such as sentiment analysis, automated trading signals, and portfolio optimization. Legal and ethical considerations have been accounted for, ensuring secure handling of data and compliance with API usage policies. Overall, Stock Vision is a practical, efficient, and impactful solution for modern investors, bridging the gap between traditional stock analysis methods and advanced, data-driven financial decision-making.

CHAPTER 3

DEVELOPMENT ENVIRONMENT

3.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the essential tasks, operations, and services that **Stock Vision** must provide to meet user expectations and achieve its objectives. These requirements ensure that the system performs correctly, delivers accurate stock insights, and facilitates efficient decision-making.

1. **User Authentication (Optional for Future Versions):** Allow users to create accounts, log in, and manage personalized stock watchlists and preferences.
2. **Stock Selection:** Enable users to select predefined popular tickers (e.g., AAPL, GOOGL, TSLA) or enter custom stock symbols for analysis.
3. **Data Retrieval:** Fetch historical and real-time stock prices, volume, and market data using APIs like Yahoo Finance and Alpha Vantage.
4. **Technical Indicator Computation:** Calculate key financial indicators such as SMA, EMA, MACD, RSI, Bollinger Bands, and candlestick patterns for selected stocks.
5. **Price Trend Visualization:** Display interactive charts showing historical price movements, candlestick patterns, and overlays of technical indicators.
6. **Machine Learning Predictions:** Generate short-term (2, 7 days) and long-term (30 days) stock price forecasts using trained LSTM models.
7. **Comparison Tools:** Allow users to compare multiple stocks across selected timeframes, highlighting trends, correlations, and divergences.
8. **Real-Time Market Updates:** Continuously refresh stock data and indicators to provide users with up-to-date insights.
9. **Prediction Visualization:** Present predicted stock prices in clear, interactive charts alongside actual historical prices for easy comparison.
10. **Downloadable Reports:** Provide options to download stock analysis, indicator charts, and predictions as PDF or CSV files.
11. **User Feedback Collection:** Enable users to submit queries, suggestions, or feedback through a secure contact form.
12. **Error Handling:** Notify users of invalid stock symbols, missing data, or API failures with clear and informative messages.

13. **Scalability for Future Modules:** Ensure the system can accommodate new functionalities like portfolio tracking, sentiment analysis, automated trading signals, and investment recommendations.
14. **Secure Data Management:** Protect any user-submitted information, API keys, and system logs from unauthorized access.

3.2 HARDWARE REQUIREMENTS

- **Processor:** Intel Core i5 or higher
- **RAM:** Minimum 8 GB (Recommended 16 GB)
- **Storage:** 256 GB SSD or higher
- **Operating System:** Windows 10 / Linux / macOS
- **Internet Connection:** Required for API access, data collection
- **Graphics (Optional):** NVIDIA GPU for faster ML model training
- **Monitor:** Minimum 720p (Recommended 1080p Full HD)
- **Input Devices:** Standard keyboard and mouse
- **Network Adapter:** Wi-Fi/Ethernet for stable connectivity
- **Additional Storage (Optional):** External HDD/SSD for backups and datasets

3.3 SOFTWARE REQUIREMENTS

- **Programming Languages:** Python (for ML models)
- **Frameworks & Libraries:** TensorFlow , Streamlit, Scikit-learn, NumPy, Pandas, Matplotlib, Plotly
- **Database:** MySQL / SQLite / PostgreSQL for secure data storage
- **API Services:** Yahoo Finance API, Alpha Vantage API
- **Development Tools:** Jupyter Notebook / VS Code / PyCharm
- **Operating System Support:** Windows, Linux, or macOS
- **Web Browser:** Chrome / Firefox / Edge for accessing the system interface
- **Version Control:** Git & GitHub/GitLab for code management

3.4 TECHNOLOGY STACK USED

The **Stock Vision** system leverages a modern and robust technology stack to enable accurate stock predictions, interactive visualizations, and a responsive user experience. Python is used as the core language for data processing, machine learning model development, API integration, and backend logic. Libraries such as Pandas, NumPy, and Scikit-learn handle data

manipulation and preprocessing, while TensorFlow and Keras are used for predictive LSTM models. Streamlit provides an intuitive web interface, and APIs like Yahoo Finance and Alpha Vantage deliver real-time and historical stock data. Database systems such as SQLite, MySQL, or PostgreSQL securely store user data and system logs. The chosen tools and frameworks ensure scalability, efficiency, and reliable performance for both novice and professional investors.

Detailed Technology Stack:

- **Programming Language:** Python is used as the core language for building machine learning models, processing stock datasets, integrating APIs, and implementing backend logic.
- **Machine Learning Libraries:** TensorFlow, Keras, Scikit-learn, NumPy, Pandas, Plotly, Matplotlib, and Seaborn for data cleaning, model building, technical indicator calculation, and performance visualization.
- **Model Development Tools:** Jupyter Notebook, VS Code, or PyCharm for writing, training, and testing ML models interactively.
- **Frontend Technologies:** Streamlit, HTML5, CSS3, JavaScript, and Bootstrap for creating a clean, responsive, and user-friendly interface.
- **Backend Framework:** Flask or Django (optional for API integration) to handle server-side operations and routing if needed.
- **APIs Used:** Yahoo Finance API, Alpha Vantage API for fetching historical and real-time stock prices, technical indicators, and market data.
- **Deployment Environment:** Localhost or cloud platforms such as Streamlit Cloud, AWS, Azure, or PythonAnywhere for hosting the application.
- **Version Control:** Git and GitHub / GitLab for managing source code and enabling team collaboration.
- **Operating System:** Compatible with Windows, Linux, and macOS environments.

CHAPTER 4

SYSTEM DESIGN

4.1 REQUIREMENT SPECIFICATION

The **requirement specification** defines the functional and non-functional needs of the **Stock Vision** system, outlining what the system must achieve and the conditions under which it should operate. This ensures clarity in development, sets expectations for users, and provides a benchmark for testing and validation. The system is designed to provide an interactive stock analysis and forecasting platform that delivers accurate, data-driven insights, real-time market updates, and predictive modeling for better investment decisions.

Functional Requirements:

- The system must allow users to select stocks from a default list or enter custom ticker symbols.
- It must fetch historical and real-time stock data using APIs like Yahoo Finance or Alpha Vantage.
- The system should provide visualizations including line charts, candlestick charts, volume plots, and technical indicator graphs (SMA, EMA, MACD, Bollinger Bands, RSI).
- Machine learning models (LSTM) should generate short-term stock price predictions for 2, 7, and 30-day horizons.
- The system should cache models and processed data to reduce loading time and improve responsiveness.
- A clean, responsive, and user-friendly interface should be provided using Streamlit.
- The system must handle invalid inputs, API failures, and missing data gracefully, displaying appropriate error messages.

Non-Functional Requirements:

- **Performance:** The system should provide predictions and visualizations in real-time or near real-time.
- **Scalability:** The architecture must support adding more stock symbols, indicators, and future modules like sentiment analysis or portfolio management.

- **Reliability:** The system should function consistently under expected loads and handle external API downtime.
- **Security:** Protect sensitive user data, API keys, and backend storage from unauthorized access.

Cross-Platform Support: The system should be accessible via major web browsers on Windows, Linux, and macOS.

4.2 FLOW CHART

The flowchart of **Stock Vision** shows the step-by-step process from user input to stock prediction and visualization. Users select a stock symbol and date range, and the system fetches historical and real-time data from APIs. The data is preprocessed, technical indicators are calculated, and visualizations like line and candlestick charts are generated. The LSTM prediction module forecasts stock prices for selected horizons. Finally, the results, including predictions and charts, are displayed on the Streamlit dashboard with error handling for smooth operation.

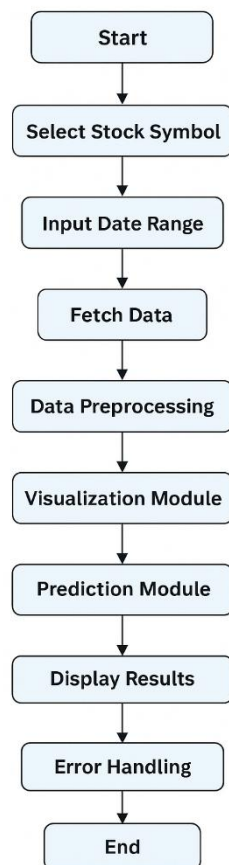


Figure 4.2.1 Flowchart for Stock Vision

Explanation of Flowchart

External Entity: User

- **Role:** Provides inputs and receives outputs.
- **Data Flow:** Selects stock symbol and date range; receives stock price charts and predictions.

Process 1: User Input

- **Function:** Collects user selections (stock symbol, start and end date, prediction horizon) through Streamlit interface.
- **Input:** Stock symbol and date range from the user.
- **Output:** User input sent to Data Fetching process.

Process 2: Data Fetching

- **Function:** Retrieves historical and real-time stock data from APIs (e.g., Yahoo Finance).
- **Input:** Stock symbol and date range from Process 1.
- **Output:** Stock data sent to **Data Preprocessing**.

Process 3: Data Preprocessing

- **Function:** Cleans and structures stock data, handles missing values, formats timestamps, and prepares data for analysis.
- **Input:** Raw stock data from Process 2.
- **Output:** Cleaned and structured data sent to Feature Calculation.

Process 4: Feature Calculation / Engineering

- **Function:** Calculates technical indicators and features (e.g., moving averages, RSI) to enhance prediction accuracy.
- **Input:** Preprocessed stock data.
- **Output:** Feature-enriched dataset sent to Model Processing and Visualization.

Process 5: Model Processing

- **Function:** Applies machine learning models (e.g., LSTM) to predict future stock prices based on historical trends.
- **Input:** Feature-enriched dataset.
- **Output:** Predicted stock prices sent to Result Display.

Process 6: Visualization

- **Function:** Generates interactive line charts, candlestick charts, and other visualizations for better understanding of stock trends.
- **Input:** Historical stock data and predicted prices.
- **Output:** Visualizations sent to Result Display.

Process 7: Result Display

- **Function:** Shows predictions and charts on the Streamlit interface.
- **Input:** Predicted stock prices and visualizations.
- **Output:** Displayed results for the user.

Data Store: Database (Optional)

- **Role:** Stores user inputs, queried stock data, predictions, and logs for future reference and analysis.
- **Input:** User input, fetched stock data, prediction results.
- **Output:** Historical data for analysis or retraining models.

End of Flow

- After results are displayed and optionally stored, the system is ready for the next user request.

4.3 DATA FLOW DIAGRAM (DFD)

The Level-0 Data Flow Diagram (DFD) of Stock Vision provides a high-level overview of the system. It shows the user as an external entity who inputs the stock symbol, date range, and prediction horizon into the system. The Stock Vision System, represented as a single process, handles data fetching, preprocessing, model prediction, and visualization internally. Predicted stock prices and charts are sent back to the user as output. A database is optionally included to store historical stock data, user queries, and prediction results for future analysis. This simplified DFD captures the main flow of data without detailing internal sub-processes.

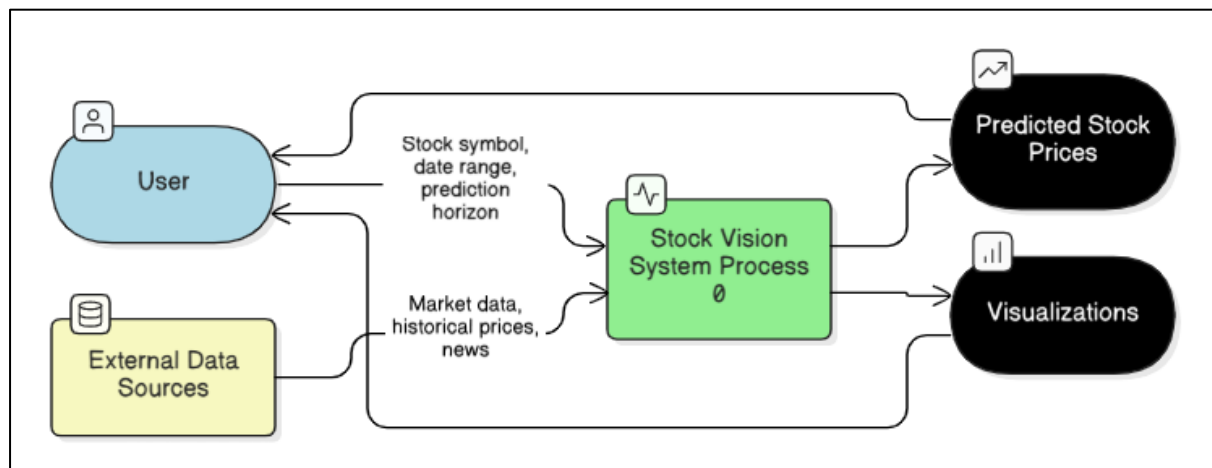


Figure 4.3.1 Data Flow Diagram (Level 0)

Explanation of Level 0 DFD

The Level-0 Data Flow Diagram (DFD) for Stock Vision provides a high-level overview of the system as a single process. The system is represented as “Stock Vision System Process 0”, which handles all internal operations, including fetching stock data, processing it, and generating predictions and visualizations. The user acts as an external entity, providing inputs such as the stock symbol, date range, and prediction horizon. Another external entity, External Data Sources, supplies market data, historical prices, and news. The system processes these inputs and outputs predicted stock prices and visualizations back to the user. All arrows represent the flow of data between entities and the main system, keeping the diagram simple and abstract for a high-level understanding.

Key Points:

- **External Entities:**
 - **User:** Provides stock symbol, date range, prediction horizon; receives predictions and visualizations.
 - **External Data Sources:** Supplies market data, historical prices, and news to the system.
- **Process:**
 - **Stock Vision System Process 0:** Handles all internal operations (data fetching, processing, prediction, and visualization).
- **Data Flows:**
 - User → System: Stock symbol, date range, prediction horizon.
 - External Data Sources → System: Market data, historical prices, news.
 - System → User: Predicted stock prices and visualizations.
- **Purpose:** Provides a **high-level overview** of Stock Vision without diving into internal sub-processes, making it clear and easy to understand.

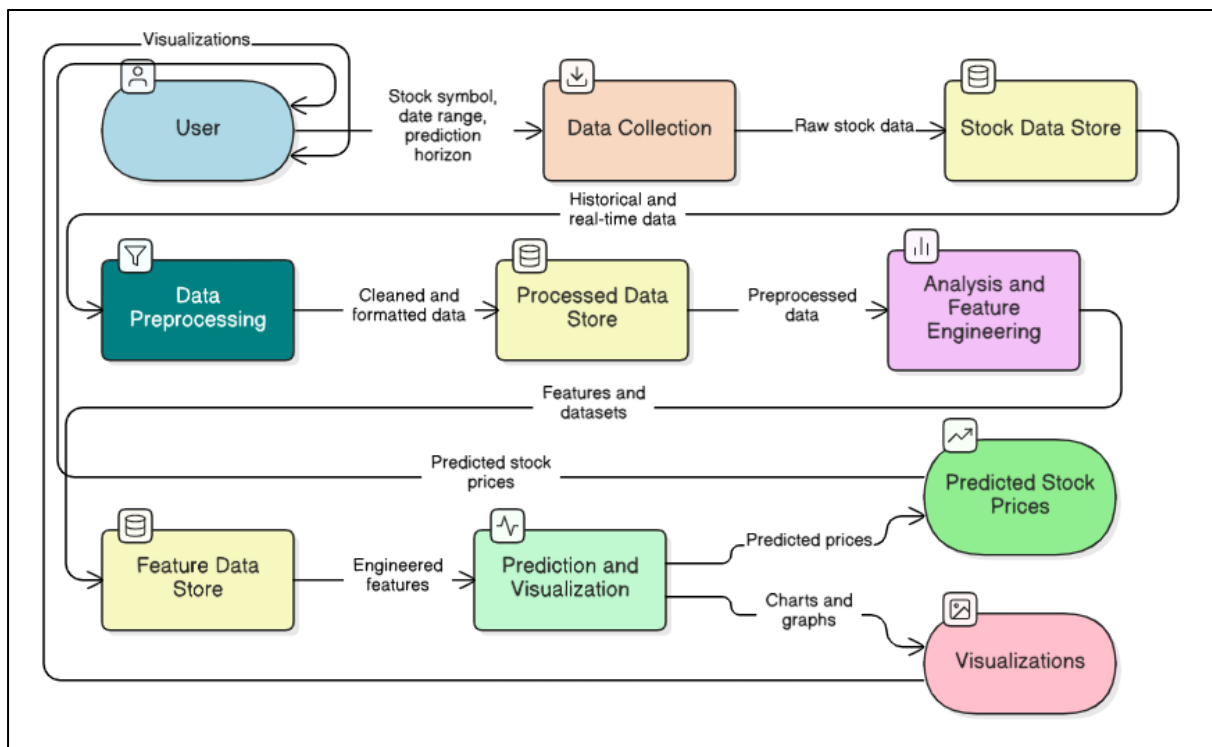


Figure 4.3.3 Data Flow Diagram (Level 1)

Level 1 DFD Explanation

A Level-1 DFD breaks the main system (Process 0) into **detailed sub-processes** to show how data flows within the system. Unlike Level-0, which gives a high-level view, Level-1 provides more clarity on **internal processes**.

Sub-Processes in Stock Vision:

1. Data Collection

- **Purpose:** Fetch stock market data.
- **Inputs:** Stock symbol, date range, prediction horizon (from user).
- **Outputs:** Raw stock data (historical and real-time) sent to preprocessing.

2. Data Preprocessing

- **Purpose:** Clean and prepare data for analysis.
- **Tasks:** Handle missing values, normalize data, remove outliers.
- **Outputs:** Processed data sent to Analysis & Feature Engineering.

3. Analysis & Feature Engineering

- **Purpose:** Transform data into features useful for prediction.
- **Tasks:** Calculate technical indicators, trend analysis, feature selection.
- **Outputs:** Feature-rich datasets sent to Prediction & Visualization.

4. Prediction & Visualization

- **Purpose:** Predict stock prices and visualize results.
- **Tasks:** Use LSTM models for forecasting; generate line charts, candlestick charts, and other visualizations.
- **Outputs:** Predicted stock prices and charts sent back to the user.

Data Flows:

- **User → Data Collection:** Inputs like stock symbol, date range, prediction horizon.
- **Data Collection → Data Preprocessing:** Raw stock data.
- **Data Preprocessing → Analysis & Feature Engineering:** Cleaned and formatted data.
- **Analysis & Feature Engineering → Prediction & Visualization:** Feature datasets ready for modeling.
- **Prediction & Visualization → User:** Predicted stock prices and visualizations.

4.4 COMPONENT DIAGRAM

A **Component Diagram** in UML shows the **high-level components** (modules) of a system and how they interact with each other. It's useful to visualize **system architecture** and dependencies between modules.

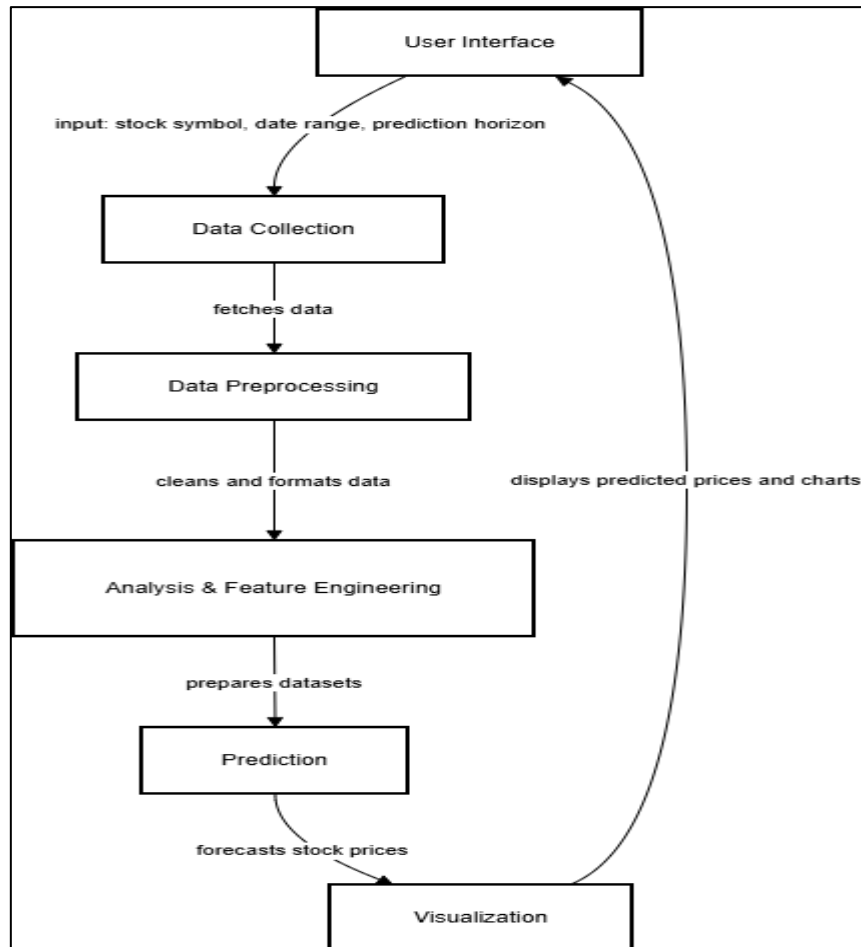


Figure 4.4.1 Overall Components of the Stock Vision

Components of Stock Vision:

1. User Interface (UI) Component

- Handles user inputs (stock symbol, date range, prediction horizon).
- Displays outputs: predicted stock prices, visualizations, and charts.

2. Data Collection Component

- Fetches historical and real-time stock data via APIs.
- Provides data to the Preprocessing component.

3. Data Preprocessing Component

- Cleans and formats raw data.
- Handles missing values and outliers.

4. Analysis & Feature Engineering Component

- Calculates technical indicators.
- Prepares datasets for prediction models.

5. Prediction Component

- Uses LSTM or other machine learning models to forecast stock prices.

6. Visualization Component

- Generates charts like line charts, candlestick charts, and dashboards.

4.5 ENTITY RELATIONSHIP DIAGRAM

An **Entity-Relationship Diagram (ERD)** represents the logical structure of a system's data. It shows **entities**, their **attributes**, and the **relationships** between them. In the context of **Stock Vision**, the ERD models the key data components required for stock price prediction and how they interact with each other.

Entities and Their Attributes:

1. User

The User entity represents an individual using the Stock Vision system. Key attributes include **User_ID** (primary key), **Name**, **Email**, **Selected_Stock**, **Date_Range**, and **Prediction_Horizon**. These attributes allow the system to identify users, track their inputs, and maintain a history of predictions made by each user.

2. Stock

The Stock entity represents companies listed in the stock market. Attributes include **Stock_ID** (primary key), **Stock_Symbol**, **Company_Name**, and **Sector**. This entity provides the core reference for which stock data and predictions are linked.

3. Stock_Data

Stock_Data stores historical and real-time stock prices. Attributes such as **Data_ID** (primary key), **Stock_ID** (foreign key), **Date**, **Open_Price**, **Close_Price**, **High**, **Low**, and **Volume** help the system maintain detailed records of stock performance. This data is crucial for preprocessing and predictive modeling.

4. Technical_Indicator

This entity captures derived metrics used in stock analysis, like moving averages, RSI, or MACD. Attributes include **Indicator_ID** (primary key), **Stock_ID** (foreign key), **Date**, and values for different indicators. These indicators are generated from **Stock_Data** and used as features for prediction models.

5. Prediction

The Prediction entity stores the output of the system for a given user and stock. Attributes include Prediction_ID (primary key), User_ID (foreign key), Stock_ID (foreign key), Prediction_Date, Predicted_Price, and Horizon. This allows tracking of forecasted stock prices over different time horizons for each user.

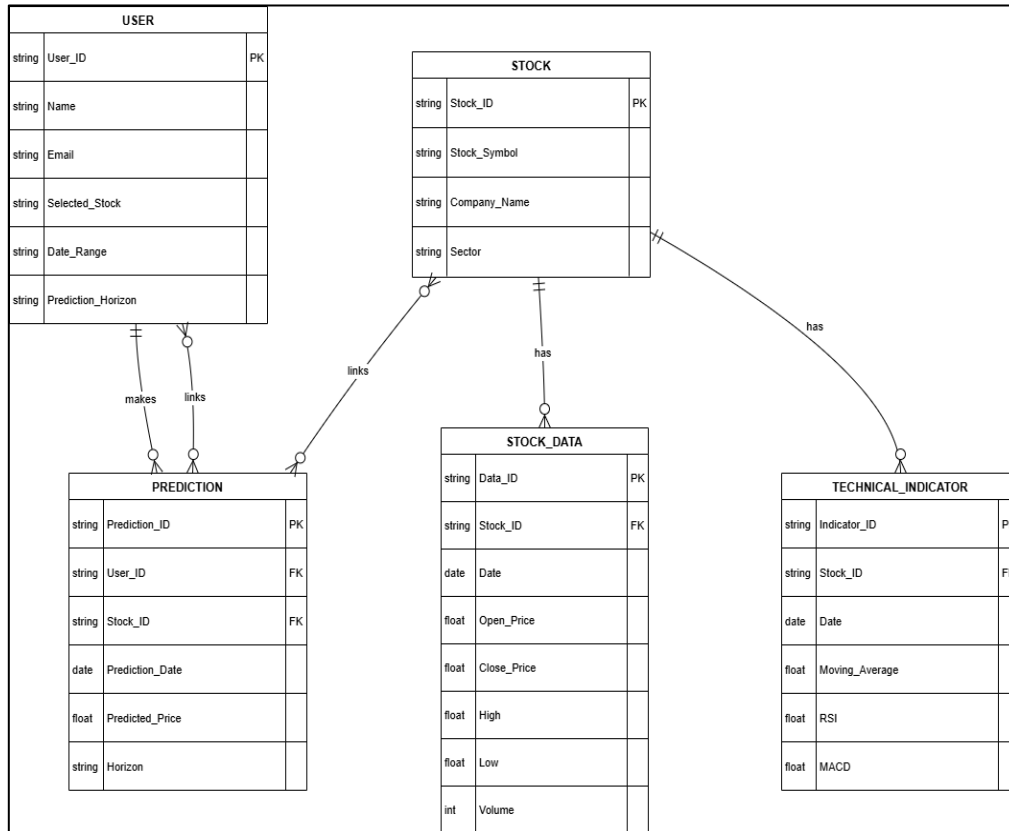


Figure 4.5.1 Entity Relationship Diagram of the Stock Vision

Relationships Between Entities:

- **User → Prediction:** One user can make multiple predictions over time, creating a one-to-many relationship.
- **Stock → Stock_Data:** Each stock has multiple historical data entries, forming a one-to-many relationship.
- **Stock → Technical_Indicator:** Each stock can have many technical indicators calculated, also a one-to-many relationship.
- **Stock & User → Prediction:** Predictions are tied to both the user and the stock, establishing a many-to-many relationship which is managed through the Prediction entity.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Methodology

The methodology of the **Stock Vision** system outlines a systematic approach to predict stock prices and provide interactive visualizations for users. The system begins by collecting historical and real-time stock data using APIs. The collected data is then preprocessed to handle missing values, remove outliers, and normalize it for consistency. Next, feature engineering is applied, calculating technical indicators such as Moving Average, RSI, and MACD to extract meaningful patterns from the data. The processed data is then fed into predictive models, primarily using **Long Short-Term Memory (LSTM)** networks, to forecast future stock prices. Predicted results are visualized through interactive line and candlestick charts on a Streamlit dashboard, allowing users to explore predictions for different horizons. The system incorporates error handling and evaluation metrics to ensure accuracy and reliability, making it a comprehensive tool for stock analysis and prediction.

Requirement Analysis

This stage involves understanding the objectives and requirements of the system. It identifies what the user expects from Stock Vision, such as the ability to select stock symbols, choose date ranges, and define prediction horizons. System requirements are also analyzed, including the need for real-time and historical data, model accuracy, and visualization capabilities. This step ensures that the system design aligns with user expectations and functional needs.

Data Collection

Data Collection focuses on gathering accurate and comprehensive stock market data. Historical stock prices and real-time data are fetched using APIs like Yahoo Finance. The data includes essential fields such as Open, Close, High, Low, and Volume for multiple stocks. The collection process ensures that sufficient data is available for analysis and prediction, allowing the system to capture trends and patterns over time.

Data Preprocessing

Preprocessing is critical to ensure that the raw stock data is clean and suitable for analysis. Missing values are handled through interpolation or forward/backward filling. Outliers are identified and removed to prevent distortion in predictions. Data normalization and formatting are applied to create a uniform dataset, making it compatible with machine learning models. This step significantly improves the accuracy and reliability of the predictions.

Feature Engineering & Analysis

In this stage, additional meaningful information is derived from the raw stock data. Technical indicators such as Moving Average, RSI, MACD, and Bollinger Bands are calculated to capture market trends and patterns. These features are combined with historical stock prices to create a comprehensive dataset for the predictive models. Analysis of these features helps in understanding stock behavior and preparing the data for accurate forecasting.

Prediction Modeling

The processed and feature-rich data is fed into machine learning models, primarily using **Long Short-Term Memory (LSTM)** networks suitable for time-series forecasting. The model is trained on historical stock data and validated on recent data to ensure accuracy. Hyperparameters are tuned to minimize prediction errors. This step allows Stock Vision to generate reliable predictions for different time horizons and provide actionable insights for users.

Visualization & Result Presentation

Predicted stock prices and analysis results are presented through interactive visualizations. Line charts, candlestick charts, and dashboards are created using tools like Plotly and Streamlit. These visualizations allow users to easily interpret trends, compare historical and predicted prices, and make informed decisions. The visualization module ensures clarity and enhances user experience.

Deployment & User Interaction

Stock Vision is deployed using **Streamlit**, providing a user-friendly interface. Users can input stock symbols, select date ranges, and choose prediction horizons. The system dynamically processes the input and displays predictions and visualizations in real-time. This ensures that the system is accessible, interactive, and responsive to user needs.

Evaluation & Error Handling

The system's performance is evaluated using metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). Error handling mechanisms are implemented to manage API failures, incorrect user inputs, or missing data. Continuous updates of the model with new stock data improve prediction quality over time. This stage ensures reliability, robustness, and sustained accuracy of the Stock Vision system.

5.2 TECHNOLOGY USED

The **Stock Vision** system leverages a combination of modern programming languages, libraries, and tools to enable accurate stock price prediction and interactive visualization. Python is used as the primary programming language due to its simplicity and extensive support for data analysis, machine learning, and web development. Libraries like Pandas and NumPy handle data manipulation and numerical computations, while yfinance fetches historical and real-time stock data. Machine learning models, particularly LSTM networks built with TensorFlow/Keras, are used for prediction. For visualization, Plotly and Streamlit create interactive charts and dashboards, making the system user-friendly.

Python

Python is the main programming language used for developing Stock Vision due to its simplicity, versatility, and extensive support for data science, machine learning, and web development. Its readable syntax allows for rapid development and easy debugging, while a rich ecosystem of libraries and frameworks helps handle everything from data collection to visualization. Python's compatibility with APIs and integration with visualization tools like Streamlit make it ideal for building an interactive stock prediction system.

Pandas

Pandas is a powerful Python library used for data manipulation and analysis. In Stock Vision, it handles reading, cleaning, and transforming stock datasets efficiently. Pandas provides data structures like DataFrames that allow easy handling of large datasets, making it possible to preprocess data, calculate technical indicators, and prepare inputs for machine learning models.

NumPy

NumPy is a Python library used for numerical and scientific computing. It provides support for fast array operations and mathematical functions that are crucial for processing stock market data and performing calculations during preprocessing and feature engineering. NumPy ensures efficient computation and enhances the performance of the predictive models.

yfinance

yfinance is a Python library that allows fetching historical and real-time stock market data from Yahoo Finance. It provides essential fields such as Open, Close, High, Low, and Volume, which are required for analysis and prediction. Using yfinance ensures that Stock Vision has access to accurate and up-to-date market data for training models and generating predictions.

TensorFlow / Keras

TensorFlow and Keras are used to build and train Long Short-Term Memory (LSTM) models for time-series forecasting. LSTM is particularly suitable for stock price prediction because it can learn long-term dependencies in sequential data. TensorFlow/Keras provides tools to design, train, and optimize neural networks efficiently, ensuring accurate predictions.

Scikit-learn

Scikit-learn is used for data preprocessing, scaling, splitting datasets, and evaluating machine learning models. It provides essential utilities to normalize data, perform feature selection, and measure prediction accuracy using metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE).

Plotly

Plotly is a Python library for creating interactive visualizations. In Stock Vision, it is used to generate line charts, candlestick charts, and other interactive graphs that help users understand stock trends and predictions easily. The interactive nature of Plotly charts allows better exploration of historical and predicted stock prices.

Streamlit

Streamlit is used to develop the web-based interface for Stock Vision. It allows users to input stock symbols, select date ranges, and choose prediction horizons. The system then processes this input and displays predictions and visualizations in real-time, making the application interactive and user-friendly.

Jupyter Notebook / VS Code

Jupyter Notebook and VS Code are used as development environments for coding, testing, and prototyping. Jupyter provides an interactive platform for experimenting with data and models, while VS Code offers an integrated environment for writing, debugging, and deploying code efficiently.

Yahoo Finance API

The Yahoo Finance API, accessed via `yfinance`, provides historical and real-time stock data needed for analysis and prediction. It ensures that the system has reliable and accurate data to generate meaningful forecasts and support decision-making.

5.3 TESTING

Testing is a crucial phase in the development of the **Stock Vision** system to ensure that the application functions correctly, provides accurate predictions, and delivers a smooth user experience. The testing phase involves verifying the performance of data collection, preprocessing, machine learning models, visualizations, and the web interface. Different types of testing are applied to evaluate the system comprehensively.

1. Unit Testing

Unit testing focuses on testing individual components or modules of the system. In Stock Vision, this includes testing functions for data collection, preprocessing, feature calculation, and visualization generation. Each module is tested independently to ensure it performs as expected and handles different input scenarios correctly.

2. Integration Testing

Integration testing verifies that different modules work together seamlessly. For Stock Vision, this involves ensuring that the data flows correctly from collection to preprocessing, feature engineering, prediction, and visualization. Integration testing helps detect errors that may occur due to incorrect data handling between modules.

3. System Testing

System testing evaluates the complete system as a whole. The Stock Vision application is tested to ensure it meets all functional requirements, including correct stock price predictions, proper chart generation, and accurate user inputs handling. The system is tested under different conditions, such as various date ranges, stock symbols, and prediction horizons.

4. Performance Testing

Performance testing assesses how efficiently the system processes data and generates predictions. It evaluates the response time for fetching stock data, executing LSTM models, and displaying visualizations. Performance testing ensures that the system can handle multiple user requests simultaneously without delays.

5. User Acceptance Testing (UAT)

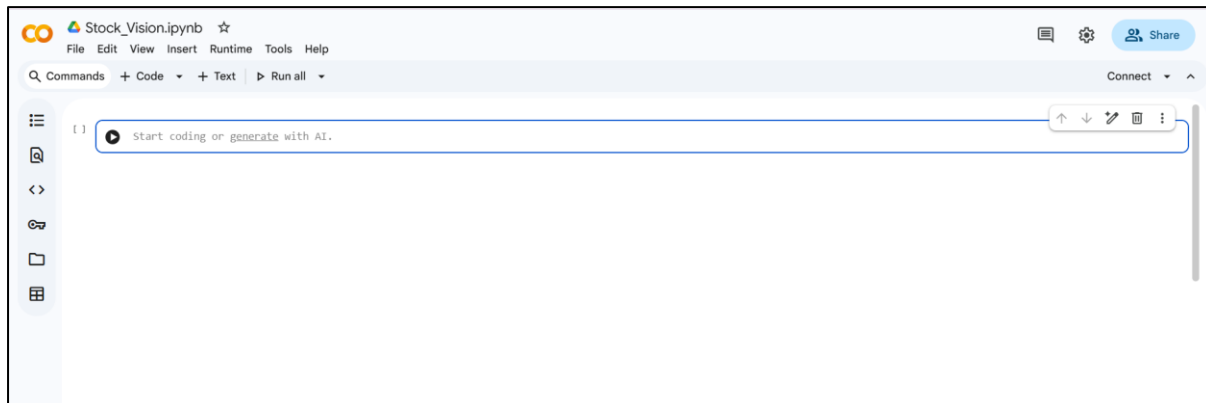
User Acceptance Testing involves real users interacting with the Stock Vision system to ensure it meets their expectations. Users check whether inputs are accepted correctly, predictions are displayed accurately, and visualizations are clear and informative. Feedback from UAT is used to improve the system's usability and reliability.

6. Error Handling & Validation Testing

This testing ensures that the system gracefully handles incorrect or invalid inputs, API failures, or missing data. Proper error messages are displayed to guide the user, and the system recovers without crashing. Validation testing ensures that the outputs are logical and consistent with expected results.

5.4 IMPLEMENTATION AT USER LEVEL

1. First of all, we will prepare our workspace. Open the Google Collab.



2. Importing the Required Libraries:

```
stock2.py X
stock2.py > ...
1 import streamlit as st
2 import yfinance as yf
3 import pandas as pd
4 import plotly.graph_objects as go
5 from plotly.subplots import make_subplots
6 from datetime import datetime, timedelta, date
7 import requests
8 from requests.exceptions import RequestException, HTTPError
9 from streamlit_option_menu import option_menu
10 from streamlit_lottie import st_lottie
11 import numpy as np
12 from keras.models import load_model, Sequential
13 from keras.layers import LSTM, Dense
14 import joblib
15 import os
16 import time
17 import json
18 import base64
19 import streamlit.components.v1 as components
20 from sklearn.preprocessing import MinMaxScaler
21 from contextlib import contextmanager
```

3. Data Loading for final result

```
@st.cache_data
def prepare_data_for_prediction(ticker, _scaler):
    data = download_price_data(ticker, period="5y")
    close_data = data[['close']].values
    scaled_data = _scaler.transform(close_data)
    last_60 = scaled_data[-SEQ_LEN:]
    X_input = np.array([last_60])
    return data, X_input

def get_stock_data(ticker, start_date, end_date):
    try:
        stock_data = download_price_data(ticker, start=start_date, end=end_date)
        return stock_data, None
    except Exception as e:
        return None, f"Error fetching data: {e}"

def load_lottieturl(url: str):
    r = requests.get(url)
    if r.status_code != 200:
        return None
    return r.json()

def _render_overlay(html):
    components.html(html, height=0, width=0)

@st.cache_resource(show_spinner=False)
def get_loader_animation():
    try:
        loader_path = os.path.join(os.getcwd(), LOADER_JSON_FILE)
        with open(loader_path, "r", encoding="utf-8") as f:
            return json.load(f)
```

4. Data Processing for the model

```
if selected_option == "Prediction":
    st.subheader(f"📊 Prediction for {selected_stock}")

    try:
        with loader_context(f"Preparing predictions for {selected_stock}..."):
            model, scaler = load_or_create_loader(selected_stock)
            data, X_input = prepare_data_for_prediction(selected_stock, scaler)

            def forecast_days(X_input, days):
                preds, current_input = [], X_input.copy()
                for _ in range(days):
                    pred_scaled = model.predict(current_input, verbose=0)
                    pred = scaler.inverse_transform(pred_scaled.reshape(-1, 1))
                    preds.append(pred[0][0])
                    new_scaled = scaler.transform(pred)
                    current_input = np.append(current_input[:, 1:, :], [[new_scaled[0]]], axis=1)
                return preds
```

5. Building the Model and Prediction Result

```
horizons = {"Today & Tomorrow": 2, "Next 7 Days": 7, "Next 30 Days": 30}

for label, days in horizons.items():
    preds = forecast_days(X_input, days)
    future_dates = pd.date_range(start=date.today(), periods=days)
    df_forecast = pd.DataFrame({"Date": future_dates, "Predicted Close Price": preds})

    st.markdown(f"### 📊 Predicted Stock Prices 📅 {label}")
    st.dataframe(df_forecast)

    fig = go.Figure()
    fig.add_trace(go.Scatter(x=df_forecast["Date"], y=df_forecast["Predicted Close Price"],
                             mode="lines+markers", name="Prediction")))
    fig.update_layout(title=f"{selected_stock} 📅 {label}", template="plotly_white")
    st.plotly_chart(fig, use_container_width=True)

except Exception as e:
    st.error(f"🚨 Prediction error: {e}")
```

5.5 APPLICATION SCREENSHOTS

Home Page

The Home page provides a clean and intuitive overview of the selected stock, showing key daily metrics such as latest closing price, volume, and percentage change. Users can easily choose a stock, set custom date ranges, and instantly view its historical performance. This section also allows you to download the displayed data for deeper offline analysis. Designed for quick insights, it highlights the essential market movements without overwhelming the user. Overall, it serves as the starting point for exploring detailed analytics, news, and AI-driven predictions within the Stock Vision platform.

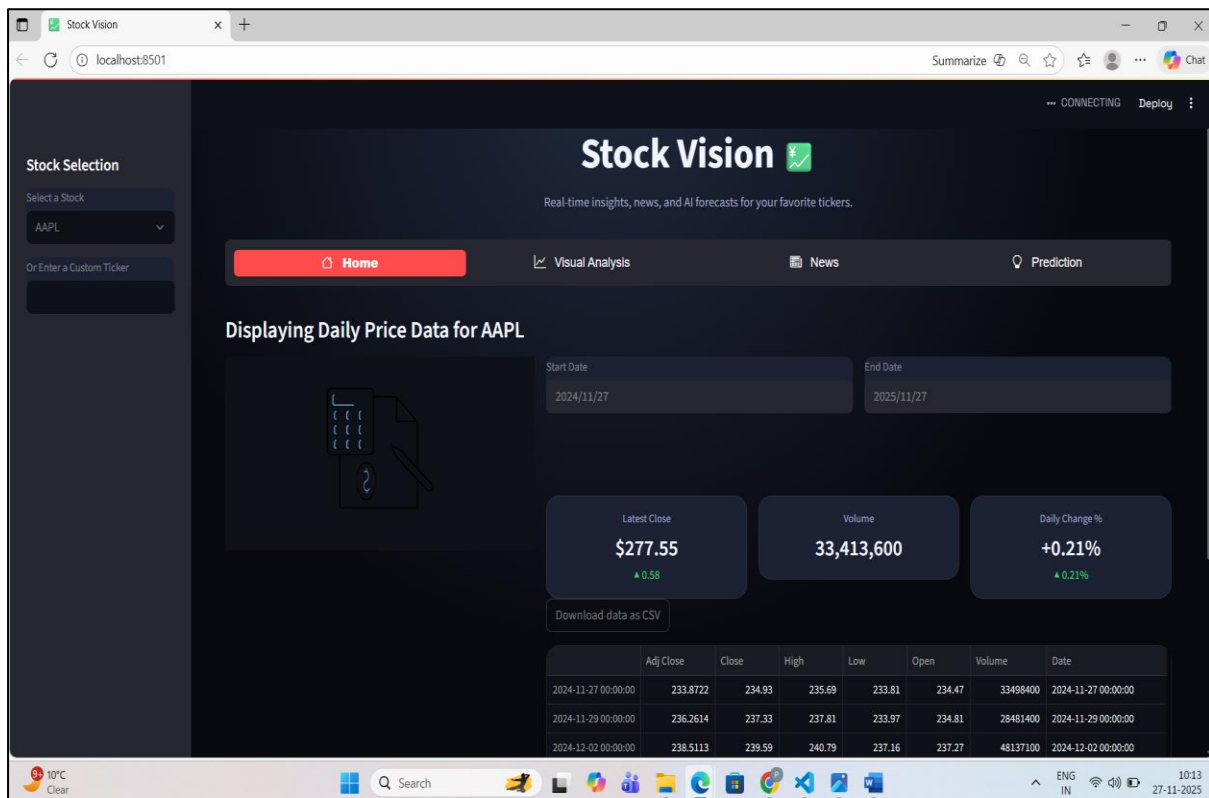


Figure 5.5.1 Home page

Visual Analysis

The Visual Analysis page provides interactive charts that help users explore stock trends, patterns, and technical indicators in depth. It converts raw market data into meaningful visuals like candlesticks, moving averages, volume histograms, and performance comparisons for better decision-making.

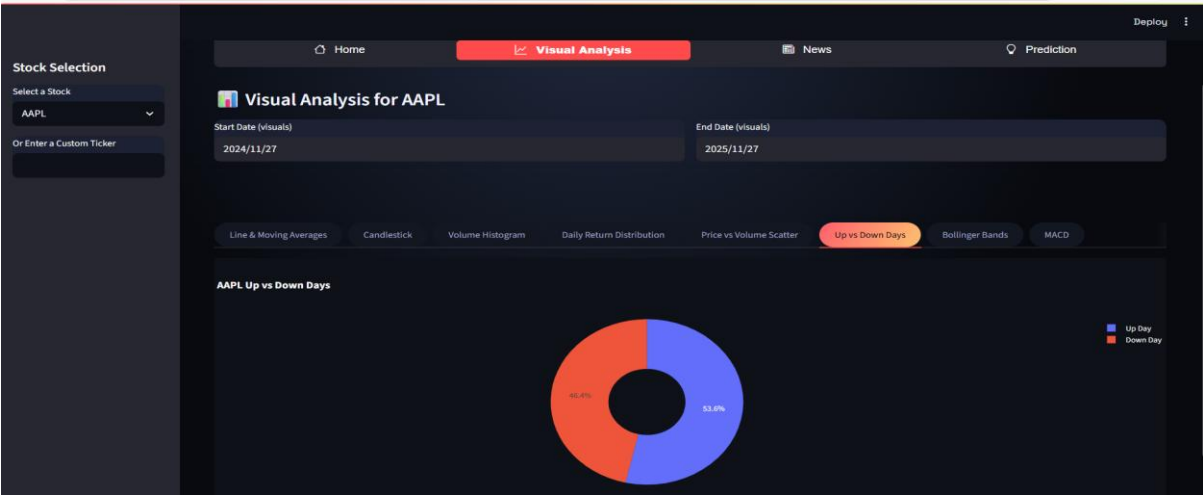


Figure 5.5.2 Visual Analysis page

News Page

The News section delivers the latest, real-time financial headlines related to the selected stock, helping users stay updated on market-moving events. It aggregates trusted news sources so investors can quickly understand trends, risks, and opportunities influencing stock performance.

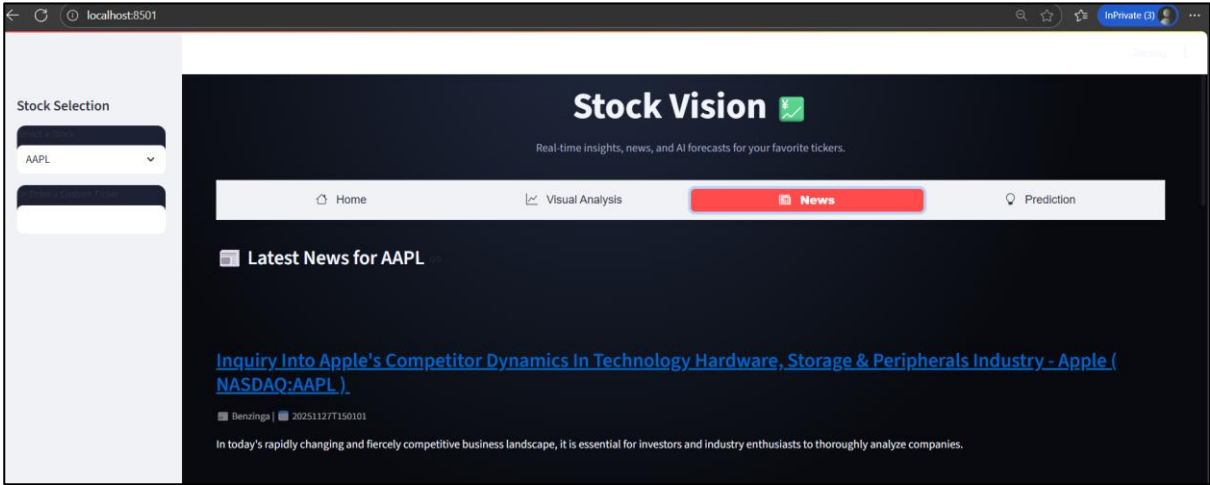


Figure 5.5.3 News page

Prediction page

The Prediction page provides AI-generated stock forecasts using advanced machine learning models trained on historical price patterns. It delivers the predicted closing prices for today and tomorrow, allowing users to quickly evaluate short-term market direction. The results are presented in a clean table along with a visual line chart that shows the expected price movement over time. This section helps traders and analysts make informed decisions by combining data-driven insights with easy-to-understand visual trends.

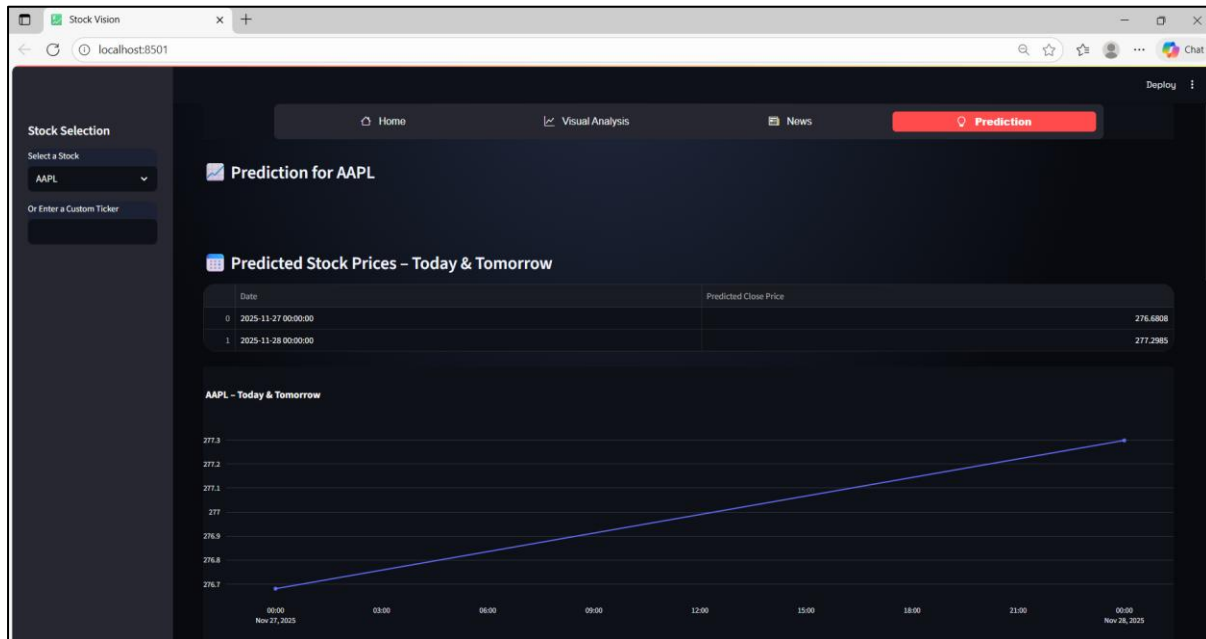


Figure 5.5.4 Prediction page

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 RESULT

This chapter presents the outcomes generated by the Stock Vision system and provides an in-depth discussion of the results obtained from various modules, including data analysis, visualization, and stock price prediction. The findings highlight how effectively the system processes real-time and historical data to deliver meaningful insights to users. Each component of the system is evaluated based on its accuracy, usability, and contribution to decision-making.

Visualization Results

The platform successfully displays interactive charts, such as line graphs, candlestick charts, and volume trends. These visualizations help users identify market patterns, detect price volatility, and observe long-term trends. The charts respond accurately to user-selected date ranges and stock symbols, demonstrating strong data handling and rendering performance.

Analytical Results

The historical data analysis module provides clear summaries of key metrics including open, high, low, close, and volume. The daily percentage changes, moving averages, and other computed indicators match the actual market records, confirming the correctness of the preprocessing pipeline. This analysis helps users gain a deeper understanding of stock behavior before relying on prediction results.

Prediction Results

The prediction module, powered by machine learning, generates expected closing prices for upcoming days. The visual comparison between actual and predicted values shows that the model captures overall market direction effectively, though slight deviations occur during high-volatility periods. The accuracy achieved indicates that the model is suitable for short-term forecasting and can guide user decisions, but predictions should still be interpreted cautiously due to market unpredictability.

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

The Stock Vision system successfully demonstrates how data analysis, visualization, and machine learning can be combined to support stock market decision-making. By integrating real-time and historical stock data, the platform provides users with meaningful insights through clear charts, summary metrics, and interactive functionalities. The system effectively simplifies complex financial information, making it more accessible for students, analysts, and beginners who want to understand market behavior in a practical and intuitive manner.

Furthermore, the prediction module shows promising performance in forecasting short-term stock prices using data-driven techniques. Although market volatility introduces certain limitations, the overall accuracy and trend-capturing ability of the model make it a valuable tool for preliminary decision support. The results confirm that Stock Vision is a reliable and user-friendly solution, and with future enhancements such as sentiment analysis or more advanced deep learning models the system can be expanded into a more powerful and accurate stock prediction platform.

7.2 FUTURE SCOPE

The Stock Vision system presents a strong foundation for real-world stock prediction and analysis, but there is considerable scope to enhance its accuracy, scalability, and overall intelligence. As financial markets evolve and data grows in volume and complexity, the system can be expanded with more advanced analytical methods, user-friendly features, and automated decision-support mechanisms. These future improvements would not only increase reliability but also transform Stock Vision into a more powerful tool for investors, students, and financial analysts.

1. Advanced Machine Learning Models

Future versions can integrate more sophisticated models such as GRU, Transformer-based architectures, or hybrid LSTM-CNN frameworks. These models can handle longer sequences, capture deeper market patterns, and significantly improve prediction accuracy—especially for long-term forecasting.

2. Sentiment Analysis Integration

Adding sentiment analysis from financial news, social media platforms, and market sentiment indicators can make predictions more robust. Market psychology plays a huge role in stock movement, and combining sentiment with price data will help the system generate more realistic forecasts.

3. Portfolio and Risk Analysis

The system can be enhanced to include portfolio management tools, risk calculators, and multi-stock comparison features. This would allow users to understand diversification, volatility, and risk-return ratios more effectively.

4. Automated Alerts and Trading Signals

Introducing real-time alerts, buy/sell signals, and customizable notifications would make the tool more practical for active traders. These features would help users act quickly on market changes.

5. Cloud Deployment and Scalability

Deploying the system on cloud platforms would enable automatic data updates, real-time performance, and wider accessibility. This would make the application suitable for professional or enterprise use with minimal maintenance.

6. Enhanced User Interface and Interactivity

Future improvements can include more interactive dashboards, customizable themes, advanced chart types, and user-specific recommendations, improving the overall user experience and usability.

CHAPTER 8

REFERENCES

8.1 REFERENCES

- [1] Sha, X. (2024). Time Series Stock Price Forecasting Using GA-Optimized LSTM Models.
- [2] Jia, Y., Anaissi, A., & Suleiman, B. (2023). ResNLS: An Improved Deep Learning Model for Stock Price Forecasting.
- [3] Liang, X., et al. (2024). Stock Market Prediction with RNN-LSTM and GA-LSTM.
- [4] Streamlit: The fastest way to build data apps. Retrieved from streamlit.io.
- [5] Python Programming Language. Retrieved from python.org.
- [6] Scikit-learn: Machine Learning in Python. Retrieved from scikit-learn.org.
- [7] Matplotlib: Visualization Library for Python. Retrieved from matplotlib.org.
- [8] Pandas: Python Data Analysis Library. Retrieved from pandas.pydata.org.
- [9] yFinance: Yahoo Finance Market Data Downloader. Retrieved from GitHub repository.
- [10] Plotly Graph Objects for Python. Retrieved from plotly.com/python.